МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 5

Работа с множествами и словарями в языке Python

Вариант 13

по дисциплине «Программирование на языке Python»

Выполнил
Тебаев Мансур Дахирович
Группа ЭНЭ-б-о-25-2
Проверил _____

Ставрополь 2026

1 Тема и цель работы

Тема — Работа с множествами и словарями в языке Python. Освоить операции над множествами, накопление и отбор данных в словарях, обработку списка записей и проверку командного интерфейса.

Во всех обязательных группах индивидуальных заданий выбран вариант 13. Количество групп: 3.

Локальный проект: python-lab-05. Подготовлен локальный Git репозиторий; удалённый адрес будет добавлен после публикации владельцем. Ссылка на удалённый репозиторий: не опубликован.

2 Среда и порядок запуска

Проверка выполнена в Windows 10, Python 3.14.6, с использованием виртуального окружения. Сторонние библиотеки для учебных программ не требуются. Числа вводятся в десятичной записи с точкой; если иное не указано, поля вводятся отдельно, каждое с новой строки. Для списков вводится одна строка чисел через пробел.

Запуск и пошаговая отладка выполнены в VS Code 1.144.0 с Python Debugger 2026.6.0 через Extension Host API. Контрольные входы поданы из подготовленных данных. Отладчик подтвердил значения переменных и следующий шаг; журнал сохранён в results/vscode_debug.json.

Из папки проекта для создания собственного окружения и запуска используются команды:

```
python -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe set_operations.py
.\.venv\Scripts\python.exe verify.py
```

Файл verify.py использует тот интерпретатор, которым он запущен. Эталонные данные лежат в fixtures.json, полные результаты — в results/checks.json и results/checks.txt. Код 0 ожидается для корректного ввода, код 1 — для проверяемых ошибок диапазона или формата. Импорт учебных модулей не запускает диалог ввода благодаря защите __name__.

3 Разобранные примеры

Операции над множествами

В универсуме U строчных латинских букв заданы A = {b, c, h, o}, B = {d, f, g, o, v, y}, C = {d, e, j, k}, D = {a, b, f, g}. Вычислить X = (A ∩ B) ∪ C и Y = (A \ D) ∪ ($\bar{C}$ \ $\bar{B}$). Дополнения берутся относительно U.

Ручной расчёт: A ∩ B = {o}; X = {d, e, j, k, o}. A \ D = {c, h, o}; $\bar{C}$ \ $\bar{B}$ = B \ C = {f, g, o, v, y}; Y = {c, f, g, h, o, v, y}. sorted используется только для вывода, вычисления выполняются над set. Пустое множество обозначается символом ∅.

Файл полного решения: python-lab-05/set_operations.py.

Исходный код:

```
import string


U = set(string.ascii_lowercase)


def calculate(a, b, c, d):
    if not (a | b | c | d) <= U:
        raise ValueError('Элемент вне универсума')
    x = (a & b) | c
    y = (a - d) | ((U - c) - (U - b))
    return x, y


def format_set(values):
    return str(sorted(values)) if values else '∅'




def main():
    a, b = set('bcho'), set('dfgovy')
    c, d = set('dejk'), set('abfg')
    x, y = calculate(a, b, c, d)
    print('X:', format_set(x))
    print('Y:', format_set(y))
```

```
if __name__ == '__main__':
    main()
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
Набор из примера	X: ['d', 'e', 'j', 'k', 'o'] Y: ['c', 'f', 'g', 'h', 'o', 'v', 'y']	X: ['d', 'e', 'j', 'k', 'o'] Y: ['c', 'f', 'g', 'h', 'o', 'v', 'y']
A=∅; B=∅; C=∅; D=∅	X: ∅ Y: ∅	X: ∅ Y: ∅
A=a; B=a; C=a; D=a	X: ['a'] Y: ∅	X: ['a'] Y: ∅
A=a; B=b; C=c; D=d	X: ['c'] Y: ['a', 'b']	X: ['c'] Y: ['a', 'b']
A=ab; B=bc; C=c; D=a	X: ['b', 'c'] Y: ['b']	X: ['b', 'c'] Y: ['b']

Команды для списка работников

Хранить список словарей с полями name — имя, post — должность, year — год приёма. Команды add, list, select N, help, exit добавляют запись, выводят список и отбирают работников со стажем строго больше N лет. Упорядочить записи по имени без учёта регистра.

Опорный год вводится первым; в проверке используется 2026. Стаж равен reference_year − year. Поля вводятся отдельными строками. Неверная запись отклоняется целиком, будущее значение года недопустимо. Команды распознаются без учёта регистра.

Файл полного решения: python-lab-05/workers.py.

Ключевой фрагмент решения:

```
workers.append({'name': name, 'post': post, 'year': year})
workers.sort(key=lambda item: item['name'].casefold())
selected = [item for item in workers
        if reference_year - item['year'] > experience]
show(selected)
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
Три работника; list; select 10	3 добавления; список по имени; select 10: Андреева	3 добавления; список по имени; select 10: Андреева
Пустая база и ошибки команд	Ошибка: нужен неотрицательный стаж Ошибка: нужен неотрицательный стаж Ошибка: неизвестная команда	Ошибка: нужен неотрицательный стаж Ошибка: нужен неотрицательный стаж Ошибка: неизвестная команда
Год приёма в будущем	Ошибка: запись отклонена Нет записей	Ошибка: запись отклонена Нет записей
bad	Ошибка: опорный год Код 1	Ошибка: опорный год Код 1

4 Общие и индивидуальные задания

Общее задание 1 Гласные

Ввести строку, определить количество русских гласных букв и множество различных встретившихся гласных. Регистр не учитывать.

casefold приводит строку к сопоставимому регистру. В список found входят все вхождения, поэтому len(found) считает повторения; set(found) удаляет повторения только для вывода различных букв. Латинские буквы гласными русского алфавита не считаются.

Файл полного решения: python-lab-05/vowels.py.

Исходный код:

```
def main():
    text = input().casefold()
    vowels = set('аеёиоуыэюя')
    found = [character for character in text if character in vowels]
    print(f'Гласных: {len(found)}')
    print('Различные:', sorted(set(found)))


if __name__ == '__main__':
    main()
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
'молоко'	Гласных: 3 Различные: ['о']	Гласных: 3 Различные: ['о']
'Ёж и Ель'	Гласных: 3 Различные: ['е', 'и', 'ё']	Гласных: 3 Различные: ['е', 'и', 'ё']
''	Гласных: 0 Различные: []	Гласных: 0 Различные: []
'aeiou'	Гласных: 0 Различные: []	Гласных: 0 Различные: []

Общее задание 2 Общие символы

Ввести две строки и вывести множество символов, встречающихся в обеих.

Пересечение set(first) & set(second) удаляет повторы. Регистр, пробелы и пунктуация учитываются. Сортировка обеспечивает воспроизводимый вывод; пустое пересечение показывается как [].

Файл полного решения: python-lab-05/common_chars.py.

Исходный код:

```
def main():
    first, second = input(), input()
    print('Общие:', sorted(set(first) & set(second)))


if __name__ == '__main__':
    main()
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
'кот!'; 'ток?'	Общие: ['к', 'о', 'т']	Общие: ['к', 'о', 'т']
'A'; 'a'	Общие: []	Общие: []
''; 'abc'	Общие: []	Общие: []
'а б!'; ' !в'	Общие: [' ', '!']	Общие: [' ', '!']

Общее задание 3 Классы школы

Создать словарь «обозначение класса → количество учеников». Изменить число учеников существующего класса, добавить новый класс, удалить класс и определить общее количество учеников.

Сначала вводится число попыток начального ввода, затем пары класс и количество. После них вводятся пара для изменения, пара для добавления и

обозначение для удаления. Повторный начальный ключ, отрицательное количество и неверная операция отклоняются без потери прежних данных. Итог выводится по возрастанию ключей.

Файл полного решения: python-lab-05/school.py.

Ключевой фрагмент решения:

```
if label not in school:
    raise ValueError
school[label] = count
# Добавление только нового класса
if label in school:
    raise ValueError
school[label] = count
# Удаление существующего класса
del school[label]
print(f'Всего учеников: {sum(school.values())}')
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
Два класса; изменить 1а; добавить 2а; удалить 1б	1а: 21 2а: 18 Всего учеников: 39	1а: 21 2а: 18 Всего учеников: 39
Пустая база; неверное изменение; добавить и удалить 2а	Ошибка: изменение отклонено Всего учеников: 0	Ошибка: изменение отклонено Всего учеников: 0
Дубликат, отрицательное и нечисловое количество	Ошибка: добавление отклонено Ошибка: удаление отклонено 1а: 20 Всего учеников: 20	Ошибка: добавление отклонено Ошибка: удаление отклонено 1а: 20 Всего учеников: 20
bad	Ошибка: число записей Код 1	Ошибка: число записей Код 1

Общее задание 4 Обратный словарь

Ввести словарь с целочисленными ключами и уникальными строковыми значениями. Построить новый словарь, в котором значения становятся ключами, а ключи — значениями.

Множество used_values предотвращает потерю записи при повторном значении. Также отклоняются повторные ключи. Все поля ошибочной попытки читаются до перехода к следующей. Пустой набор даёт два пустых словаря.

Файл полного решения: python-lab-05/invert_dict.py.

Ключевой фрагмент решения:

```
if not value or key in mapping or value in used_values:
    raise ValueError
mapping[key] = value
used_values.add(value)
inverse = {value: key for key, value in mapping.items()}
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
1→один, 2→два	Исходный: {1: 'один', 2: 'два'} Обратный: {'один': 1, 'два': 2}	Исходный: {1: 'один', 2: 'два'} Обратный: {'один': 1, 'два': 2}
Нет пар	Исходный: {} Обратный: {}	Исходный: {} Обратный: {}
Повтор ключа, значения и неверный ключ	Ошибка: пара отклонена Ошибка: пара отклонена Ошибка: пара отклонена Исходный: {1: 'a'} Обратный: {'a': 1}	Ошибка: пара отклонена Ошибка: пара отклонена Ошибка: пара отклонена Исходный: {1: 'a'} Обратный: {'a': 1}
bad	Ошибка: число записей Код 1	Ошибка: число записей Код 1

Индивидуальное задание 1 вариант 13

U — все строчные латинские буквы. A = {a, b, g, k, m, p}; B = {b, e, f, l, r}; C = {k, l, w, x}; D = {e, j, o, p, q, u, v}. Вычислить X = (A \ B) ∩ (C ∪ D); Y = (Ā ∩ B̄) \ (C ∪ D). Исходные множества не изменять.

Ручной расчёт: A \ B = {a, g, k, m, p}; C ∪ D = {e, j, k, l, o, p, q, u, v, w, x}; X = {k, p}. По закону де Моргана Y = U \ (A ∪ B ∪ C ∪ D) = {c, d, h, i, n, s, t, y, z}. calculate принимает четыре множества, проверяет универсум и возвращает новые множества.

Файл полного решения: python-lab-05/individual_01.py.

Исходный код:

```
import string


U = set(string.ascii_lowercase)


def calculate(a, b, c, d):
    if not (a | b | c | d) <= U:
        raise ValueError('Элемент вне универсума')
    x = (a - b) & (c | d)
    y = ((U - a) & (U - b)) - (c | d)
    return x, y


def format_set(values):
    return str(sorted(values)) if values else 'Ø'



def main():
    a, b = set('abgkmp'), set('beflr')
    c, d = set('klwx'), set('ejopquv')
    x, y = calculate(a, b, c, d)
    print('X:', format_set(x))
    print('Y:', format_set(y))
```

```
if __name__ == '__main__':
    main()
```

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
Вариант 13	X: ['k', 'p'] Y: ['c', 'd', 'h', 'i', 'n', 's', 't', 'y', 'z']	X: ['k', 'p'] Y: ['c', 'd', 'h', 'i', 'n', 's', 't', 'y', 'z']
A=∅; B=∅; C=∅; D=∅	X: ∅ Y: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']	X: ∅ Y: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
A=a; B=a; C=a; D=a	X: ∅ Y: ['b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']	X: ∅ Y: ['b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
A=a; B=b; C=c; D=d	X: ∅ Y: ['e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']	X: ∅ Y: ['e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
A=ab; B=a; C=bc; D=c	X: ['b'] Y: ['d', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']	X: ['b'] Y: ['d', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']

Индивидуальное задание 2 вариант 13

Вводятся название кафе и сумма заказа в целых копейках. Построить словарь «кафе → общая выручка», суммируя повторные названия. Ввести неотрицательный порог t, отобрать кафе с выручкой строго больше t и найти все кафе с наибольшей выручкой.

Число n задаёт количество попыток ввода. Название после strip непустое, регистр учитывается, ведущие нули сохраняются. Ошибочная запись пропускается, неверный n или порог завершает программу. get накапливает сумму. Отбор создаёт отдельный словарь. Все лидеры определяются по всему накопленному словарю. Деньги хранятся целыми копейками, divmod форматирует рубли без ошибок float. Пустой ввод разрешён.

Файл полного решения: python-lab-05/individual_02.py.

Ключевой фрагмент решения:

```
revenue[cafe] = revenue.get(cafe, 0) + amount
selected = {cafe: total for cafe, total in revenue.items()
        if total > threshold}
maximum = max(revenue.values())
leaders = sorted(cafe for cafe, total in revenue.items()
            if total == maximum)
```

Поиск maximum выполняется только при непустом revenue. Полные проверки ввода, цикл чтения и вывод денежных сумм находятся в указанном файле.

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	**Ожидается**	**Получено**
А:100+200; Б:250; t=250 коп.	Б: 2.50 руб. Выше порога: А: 3.00 руб. Максимум: А | 3.00 руб.	Б: 2.50 руб. Выше порога: А: 3.00 руб. Максимум: А | 3.00 руб.
Выручка равна порогу 100 коп.	А: 1.00 руб. Выше порога: Совпадений нет Максимум: А | 1.00 руб.	А: 1.00 руб. Выше порога: Совпадений нет Максимум: А | 1.00 руб.

Ввод или сценарий	Ожидается	Получено
Два лидера по 999 коп.; t=0	Выше порога: А: 9.99 руб. Б: 9.99 руб. Максимум: А, Б | 9.99 руб.	Выше порога: А: 9.99 руб. Б: 9.99 руб. Максимум: А, Б | 9.99 руб.
Нет заказов; t=0	Выручка: Выше порога: Совпадений нет Максимум: Нет данных	Выручка: Выше порога: Совпадений нет Максимум: Нет данных
Три неверных заказа; кафе 001:5 коп.	001: 0.05 руб. Выше порога: 001: 0.05 руб. Максимум: 001 | 0.05 руб.	001: 0.05 руб. Выше порога: 001: 0.05 руб. Максимум: 001 | 0.05 руб.
bad	Ошибка: число заказов Код 1	Ошибка: число заказов Код 1
0; bad	Ошибка: формат порога Код 1	Ошибка: формат порога Код 1

Индивидуальное задание 3 вариант 13

Хранить список словарей: name — строка с фамилией и именем, phone — строка цифр телефона длиной не менее трёх, birth — список трёх целых чисел [день, месяц, год]. После добавления сортировать список по первым трём цифрам телефона. Команда select ФАМИЛИЯ выводит все записи с указанной фамилией без учёта регистра. Предусмотрены также add, list, help, exit.

В name не менее двух слов; фамилия — первое слово. При выводе каждая запись имеет номер и подписи полей. Телефон проверяется как последовательность ASCII цифр и остаётся строкой, поэтому ведущие нули сохраняются. Дата вводится строго в формате dd.mm.yyyy и проверяется datetime.date, год не позже опорного. Поля ошибочной записи читаются

полностью, сама запись не добавляется. Устойчивость сортировки сохраняет порядок равных трёхзначных префиксов. При отсутствии записей выводится «Нет записей».

Файл полного решения: python-lab-05/individual_03.py.

Ключевой фрагмент решения:

```
people.append({'name': name, 'phone': phone, 'birth': birth})
people.sort(key=lambda item: item['phone'][:3])
selected = [item for item in people
        if item['name'].split()[0].casefold() == argument.casefold()]
show(selected)
```

Разбор и календарная проверка даты выполняются функцией parse_birth. Добавление расположено после успешной проверки всех трёх полей.

Основные результаты проверки. Полный журнал ввода, stdout, stderr и кодов завершения: results/checks.txt.

Ввод или сценарий	Ожидается	Получено
Пустая база и неполная команда	Нет записей Нет записей Ошибка: неизвестная команда add, list, select ФАМИЛИЯ, help, exit	Нет записей Нет записей Ошибка: неизвестная команда add, list, select ФАМИЛИЯ, help, exit
Два Иванова; поиск без учёта регистра	Добавлено Добавлено 1. Имя: иванов Пётр | Телефон: 12345 | Дата рождения: 02.02.2001 2. Имя: Иванов Иван | Телефон: 99900 | Дата рождения: 01.01.2000	Добавлено Добавлено 1. Имя: иванов Пётр | Телефон: 12345 | Дата рождения: 02.02.2001 2. Имя: Иванов Иван | Телефон: 99900 | Дата рождения: 01.01.2000

Ввод или сценарий	Ожидается	Получено
Нули в телефонах; равные префиксы; високосный год	1. Имя: Волков Виктор | Телефон: 00077 | Дата рождения: 01.01.2000 2. Имя: Белов Борис | Телефон: 00199 | Дата рождения: 29.02.2000 3. Имя: Андреев Андрей | Телефон: 00101 | Дата рождения: 01.03.2000	1. Имя: Волков Виктор | Телефон: 00077 | Дата рождения: 01.01.2000 2. Имя: Белов Борис | Телефон: 00199 | Дата рождения: 29.02.2000 3. Имя: Андреев Андрей | Телефон: 00101 | Дата рождения: 01.03.2000
Короткий телефон, неверная дата, будущий год, формат даты	Ошибка: запись отклонена Ошибка: запись отклонена Нет записей	Ошибка: запись отклонена Ошибка: запись отклонена Нет записей
Нет фамилии; неизвестная команда	Добавлено Нет записей Ошибка: неизвестная команда	Добавлено Нет записей Ошибка: неизвестная команда
bad	Ошибка: опорный год Код 1	Ошибка: опорный год Код 1

5 Трассировка и воспроизведение в редакторе

Контрольный сценарий workers.py использует опорный год 2026. Добавлены Белов Б.Б. (2016), Андреева А.А. (2015), Волков В.В. (2026). После сортировки имён и команды select 10 сравнения имеют вид:

Работник	Год	Стаж	Стаж > 10	Отбор
Андреева А.А.	2015	11	Да	Включена
Белов Б.Б.	2016	10	Нет	Исключён
Волков В.В.	2026	0	Нет	Исключён

На строке условия при Белове reference_year = 2026, item["year"] = 2016, experience = 10. Следующий шаг проверяет 10 > 10 и не включает запись. Трассировка и запуск подтвердили, что selected содержит только Андрееву. Данные сохранены в results/trace_workers.json.

Множества содержат уникальные хешируемые элементы, порядок для алгоритма не предполагается. Словарь связывает уникальные ключи с данными; присваивание существующему ключу заменяет значение, а get позволяет явно накопить сумму. Для поиска нескольких лидеров сначала определяется экстремальное значение, затем выбираются все равные ему ключи. При копировании списка словарей нужно учитывать общие вложенные записи.

В VS Code остановка на строке 51 workers.py при обработке Белова показала reference_year = 2026, experience = 10 и item[«year»] = 2016. После шага запись не включена в отбор. Итоговый вывод содержит только Андрееву со стажем 11 лет. Все проверки verify.py также успешно выполнены запуском из VS Code. Журнал остановок и вывода сохранён в results/vscode_debug.json.

6 Вывод

Разработаны и проверены 9 учебных программ. Успешно пройдены все 43 проверок, включая предусмотренные ошибочные входы. Подтверждены граничные условия, порядок обработки данных и соответствие фактических результатов ожидаемым. Закреплены операции над множествами, накопление выручки в словаре и поиск записей с сохранением типов и связей полей.

7 Источники

Методические материалы. Лабораторная работа № 5. https://disk.yandex.ru/d/yT9RQ8LG9aQ_UA

Python 3.14. Стандартные типы: https://docs.python.org/3.14/library/stdtypes.html

Python 3.14. Календарная проверка даты: https://docs.python.org/3.14/library/datetime.html